

Informe técnico y módulo Clientes

Revisión de la arquitectura de la SPA PharmaSoft

Estudiante: Josias Pari | Repositorio: github.com/Pari820/pharma-frontend
Pull Request: <https://github.com/Pari820/pharma-frontend/pull/1>

Objetivo: implementar el CRUD de Clientes reutilizando la arquitectura por capas de PharmaSoft y demostrar, con evidencias del código, que la organización de core, shared, layout y features permite agregar un módulo nuevo sin reescribir Categorías.

B1. Estructura del proyecto

```
src/app
|-- core/
|   |-- config/menu.ts
|   |-- models/{error-response.ts, pagina-response.ts}
|   |-- utils/http-error.ts
|-- features/
|   |-- categorias/{models, services, pages, categorias.routes.ts}
|   |-- clientes/{models, services, pages, clientes.routes.ts}
|   |-- inicio/
|-- layout/{header, sidebar, main-layout}
|-- shared/pages/no-encontrado/
|-- app.config.ts
|-- app.routes.ts
```

core concentra configuración, modelos transversales y utilidades; shared guarda piezas reutilizables que no pertenecen a un negocio; layout mantiene encabezado, sidebar y el contenedor estable; features encapsula cada módulo de negocio. Clientes replica el patrón de Categorías en una carpeta independiente. Evidencia: pagina-response.ts define PaginaResponse<T> (líneas 1-8) y app.routes.ts solo agrega la ruta lazy de Clientes (líneas 23-29).

B2. Mapa de rutas

Ruta	Componente	Padre	Carga	Título
/inicio	Inicio	MainLayout	loadComponent	Inicio
/categorias	CategoriaList	MainLayout	loadChildren -> component	Categorías
/categorias/nuevo	CategoriaForm	MainLayout	feature lazy	Nueva categoría
/categorias/:id/editar	CategoriaForm	MainLayout	feature lazy	Editar categoría
/clientes	ClienteList	MainLayout	loadChildren -> component	Clientes
/clientes/nuevo	ClienteForm	MainLayout	feature lazy	Nuevo cliente
/clientes/:id/editar	ClienteForm	MainLayout	feature lazy	Editar cliente
/**	NoEncontrado	-	loadComponent	Página no encontrada

B3. Responsabilidades y separación de funciones

Pieza	Tipo	Responsabilidad	Qué no hace
ClienteService	Servicio	HTTP tipado a /api/v1/clientes; paginación y CRUD.	No muestra UI ni navega.
ClienteList	Componente	Carga página, ordena, filtra, pagina y confirma baja lógica.	No usa HttpClient ni valida formularios.
ClienteForm	Componente	Formulario reactivo, validación, DTO, alta/edición y navegación.	No lista ni pagina; no conoce URL API.
CategoriaService	Servicio	Concentra las cinco operaciones HTTP de Categorías.	No muestra mensajes ni manipula la tabla.
CategoriaList	Componente	Lista, filtra y elimina con confirmación.	No crea formularios ni usa HttpClient.
CategoriaForm	Componente	Registra/edita con validaciones y mapea opcionales.	No administra listado ni URL API.
Header	Layout	Marca, enlaces globales y evento para colapsar menú.	No conoce Clientes/Categorías ni hace HTTP.
Sidebar	Layout	Renderiza MENU y resalta ruta activa.	No define endpoints ni lógica de negocio.
MainLayout	Layout	Compone Header, Sidebar y router-outlet; controla colapso.	No procesa formularios ni datos del backend.

Evidencia de código

```

ClienteService (líneas 13-28)
private readonly http = inject(HttpClient);
private readonly url = `${environment.apiUrl}/clientes`;
listar(...) {
  const params = new HttpParams()...
  return this.http.get<PaginaResponse<Cliente>>(...)
}

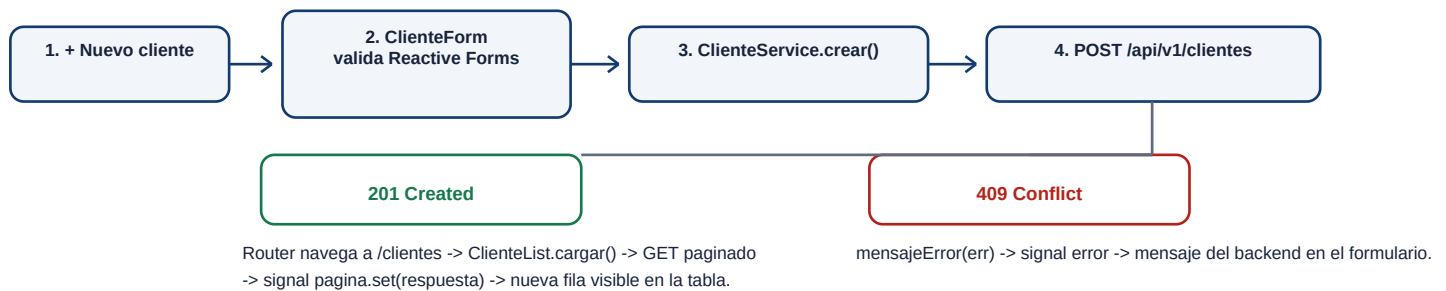
```

```

ClienteList (líneas 33-49)
clientesFiltrados = computed(() => {
  const clientes = this.pagina()?.contenido ?? [];
  return clientes.filter(cliente =>
    cliente.dni.includes(texto) ||
    nombreCompleto.includes(texto));
});

```

B4. Flujo de la operación "Registrar cliente"



B5. Preguntas de revisión

1. Archivos existentes modificados

Solo se modificaron `app.routes.ts` y `core/config/menu.ts`; el resto del módulo se agregó como archivos nuevos en `features/clientes` y `core/models/pagina-response.ts`. Ese número bajo muestra acoplamiento reducido: el crecimiento ocurre por extensión, no reescritura.

2. ¿Por qué `ClienteList` no usa `HttpClient`?

`ClienteList` inyecta `ClienteService` (`cliente-list.ts:20`) y llama `listar()` (60-66). `HttpClient` y `environment.apiUrl` están encapsulados en `cliente-service.ts:13-14`. Si cambia la URL, se ajusta `environment` o el servicio sin tocar la UI.

3. Ventaja de `loadChildren`

`app.routes.ts:23-29` carga Clientes con `loadChildren`; por eso el código se separa del bundle inicial y solo se descarga al navegar al módulo. `ng build` generó el chunk independiente "clientes-routes | 14.12 kB".

4. ¿Por qué `Header` y `Sidebar` permanecen?

`MainLayout` contiene `app-header` y `app-sidebar` fuera de `router-outlet` (`main-layout.html:1-7`). Al pasar de Categorías a Clientes solo cambia el contenido del outlet; el layout padre permanece montado.

5. Si algunos usuarios no deben ver Clientes

El punto central es `core/config/menu.ts`, porque `Sidebar` itera sobre `MENU` (`sidebar.ts:12` y `sidebar.html:4-8`). Allí puede añadirse metadata de permisos o filtrar la opción. Para impedir acceso directo también correspondería un guard de ruta.

6. Código duplicado y posible reutilización

Se repiten estados de carga/error, confirmación, patrón `list/form` y métodos CRUD en Categorías y Clientes. Puede extraerse un `CrudService` genérico, helpers compartidos y componentes de tabla/formulario configurables, manteniendo validaciones específicas por feature.

7. ¿Por qué la búsqueda solo filtra la página actual?

El `computed` toma `this.pagina()?.contenido` (`cliente-list.ts:33-35`), que contiene solo la página recibida. Para buscar todos, el backend debe aceptar un término (DNI/nombre/apellido) y filtrar antes de paginar; `ClienteService` tendría que enviar ese parámetro en `HttpParams`.

Evidencia de compilación y lazy loading

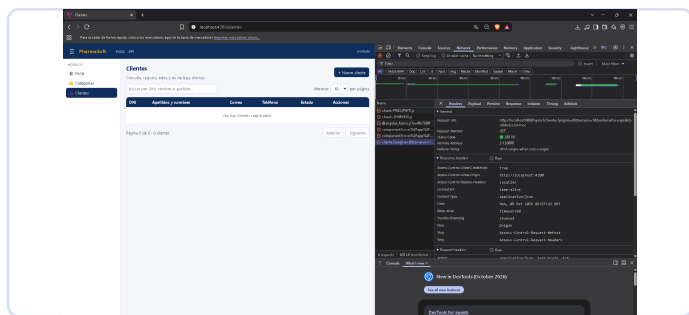
La compilación finalizó sin errores y produjo un chunk independiente para Clientes.

clientes-routes | 14.12 kB

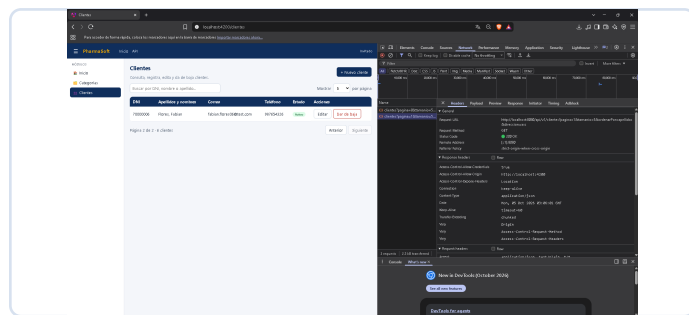


Capturas con Network/Red visible según corresponde. Pruebas 1 a 6.

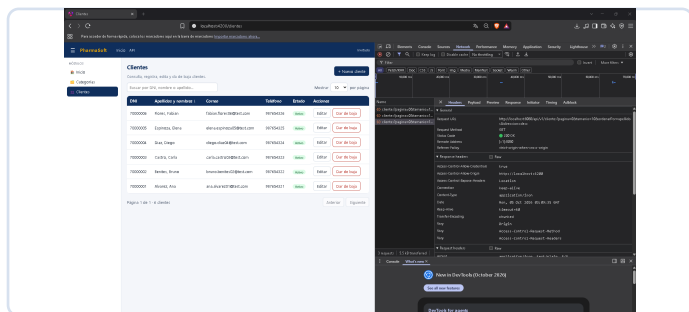
Prueba 1. Listado paginado GET 200



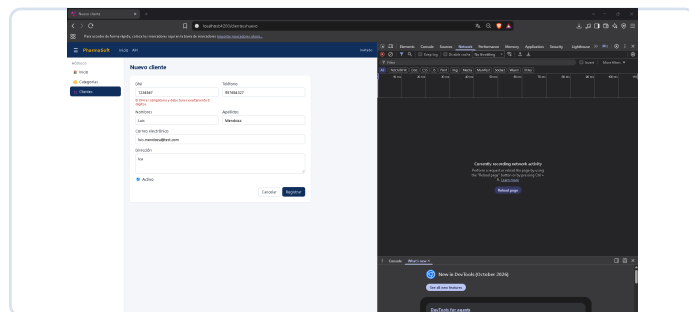
Prueba 2. Tamaño 5 y página 2



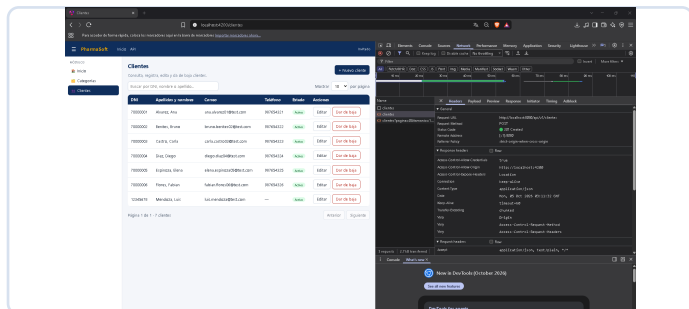
Prueba 3. Orden apellidos DESC



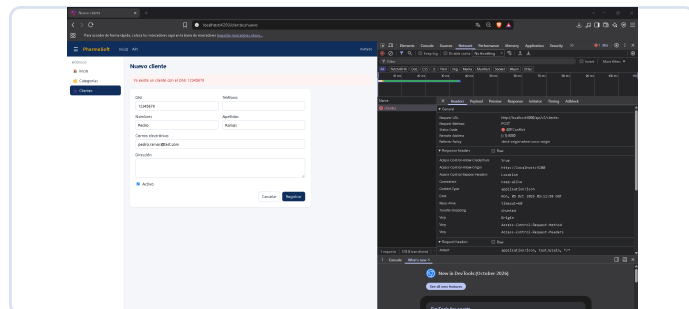
Prueba 4. DNI de 7 dígitos: validación local



Prueba 5. Registro válido sin teléfono: POST 201

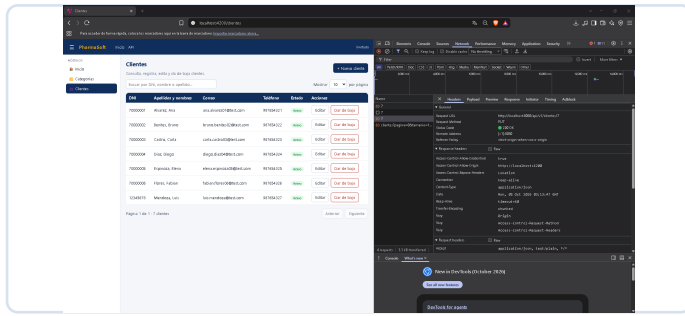


Prueba 6. DNI duplicado: POST 409

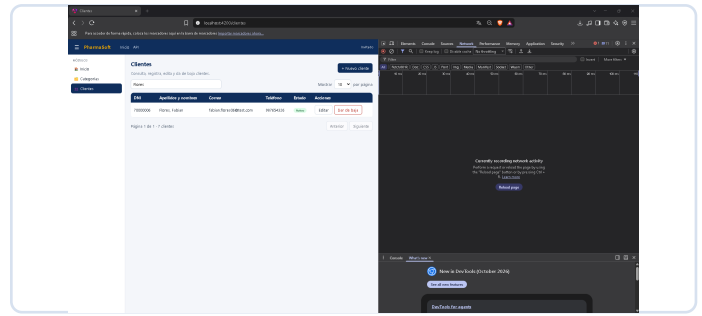


Pruebas 7 a 11 y datos de versionamiento.

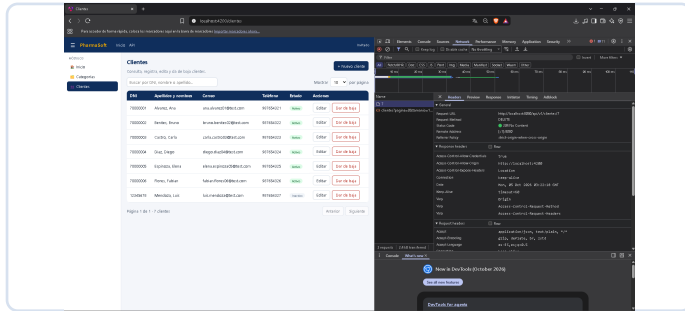
Prueba 7. Editar teléfono: PUT 200



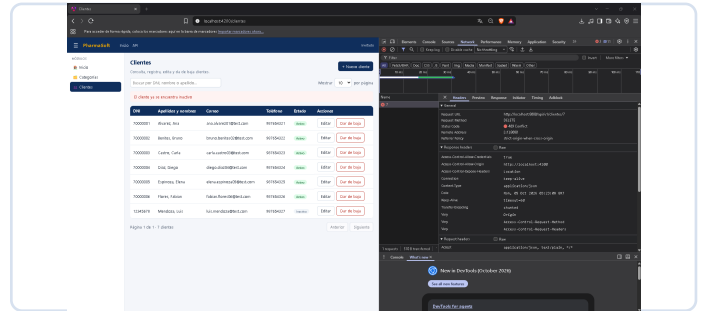
Prueba 8. Búsqueda local sin nuevas peticiones



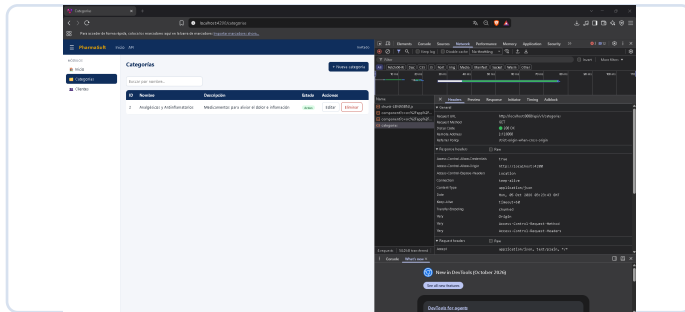
Prueba 9. Baja lógica: DELETE 204 -> Inactivo



Prueba 10. Baja repetida: DELETE 409



Prueba 11. Categorías sigue funcionando: GET 200



Versionamiento y entrega

Rama: feature/clientes-pari

Commits de código:

- feat(clientes): listar clientes con búsqueda y paginación
- feat(clientes): registrar y editar con validaciones

Pull Request abierto hacia develop:

<https://github.com/Pari820/pharma-frontend/pull/1>

Informe en repositorio: docs/informe-arquitectura.pdf

Archivo de aula virtual:

Pari_LP2_S7_Autonomia.pdf

Resultado: el módulo Clientes quedó integrado al menú y rutas, mantiene Categorías sin cambios y cumple paginación, validaciones, edición, búsqueda, baja lógica y manejo de 409.